

L7a: Dimensionality Reduction

Covariance, singular value decomposition, and low-rank approximation
CHEME 5800 Cornell University

Objectives for Today

How can we compress high-dimensional data while keeping its most important structure?

Today's concepts

- **Analyze the covariance matrix**: compute it and interpret its eigenvectors as principal directions and its eigenvalues as the variance along them.
- **Apply the singular value decomposition**: factor a matrix in full and compact form, and truncate it for the best low-rank approximation.
- **Connect PCA and the SVD**: explain how principal component analysis follows from the covariance eigendecomposition and matches the SVD of the centered data.

Lecture and Example

Lecture: Dimensionality Reduction

In this lecture, we build the empirical covariance matrix and its eigendecomposition, then factor any matrix with the singular value decomposition and truncate it.

Example: Fun with Singular Value Decomposition

In this example, we decompose a grayscale photograph into rank-one frames, measure how close the first few frames come to the original, and check the rank of their sum.

Dimensionality reduction compresses high-dimensional data while preserving its most important structure and information.

Dimensionality Reduction Problem

Suppose we have a dataset of n samples, $\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$, where each $\mathbf{x}_i \in \mathbb{R}^m$ is an m -dimensional feature vector. We want to compress each sample into k dimensions:

$$\mathbf{x}_i \in \mathbb{R}^m \longrightarrow \mathbf{y}_i \in \mathbb{R}^k, \quad k \ll m.$$

The compressed vectors should keep the most important structure and information in the data.

High-dimensional data is everywhere in modern machine learning, from genomics datasets with thousands of features to images with millions of pixels.

Why Does Dimensionality Reduction Matter?

Application	What the reduced representation captures
Genomics	Expression data can have more than 20,000 genes but only hundreds of samples; a few key patterns reveal biological processes.
Image compression	A megapixel image has millions of dimensions, but most of its information sits in far fewer components.
Recommendation systems	Low-dimensional representations of large, sparse user-item matrices capture latent preferences. The winning Netflix Prize solution used matrix factorization.
Visualization	We cannot plot 1000-dimensional data, but we can plot the top two or three components to spot patterns and clusters.

Composite Features

Each entry of the lower-dimensional vector $\mathbf{y}_i$ is a **composite feature**: a linear combination of the original features.

Reducing the dimension lets us:

- visualize high-dimensional data in two or three dimensions,
- reduce the computational cost of a machine learning algorithm, and
- store a compact representation that keeps the most important information.

Which linear combinations keep the most information? We answer this question with a linear transformation of the centered data.

A Linear Transformation

Suppose a transformation matrix $\mathbf{P} \in \mathbb{R}^{k \times m}$ maps each sample to its composite features:

$$\mathbf{y} = \mathbf{P} (\mathbf{x} - \bar{\mathbf{x}}), \quad y_i = \phi_i^\top (\mathbf{x} - \bar{\mathbf{x}}), \quad i = 1, 2, \dots, k,$$

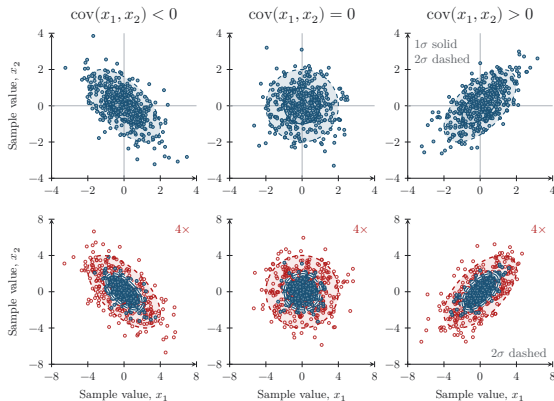
where $\bar{\mathbf{x}}$ is the mean of the original features and $\phi_i^\top$ is row i of $\mathbf{P}$. Each row extracts one component.

What are these transformation vectors ϕ_i ?

- They are the top- k eigenvectors of the data's covariance matrix. This procedure is **principal component analysis (PCA)**.
- Equivalently, they are the first k right singular vectors of the centered data matrix, computed with the **singular value decomposition (SVD)**.

The covariance matrix tells us which directions capture the most variance, so we review it first.

Sign and Scale of Covariance



The tilt of each ellipse shows the **sign** of the covariance. Multiplying the covariance matrix by four doubles the standard deviations and leaves the correlation unchanged.

Empirical Covariance Matrix

Suppose we have n samples of m features, and let $x_k^{(i)}$ be feature i of sample k . The **empirical covariance matrix** $\hat{\Sigma} \in \mathbb{R}^{m \times m}$ is a square symmetric matrix that summarizes the covariances between pairs of features. Its entries are:

$$\hat{\Sigma}_{ij} = \frac{1}{n-1} \sum_{k=1}^n (x_k^{(i)} - \bar{x}_i)(x_k^{(j)} - \bar{x}_j) \implies \boxed{\hat{\Sigma}_{ij} = \sigma_i \sigma_j \rho_{ij}}$$

where $\bar{x}_i$ is the mean of feature i over the n samples, $\sigma_i = \sqrt{\hat{\Sigma}_{ii}}$ is its standard deviation, and $\rho_{ij} \in [-1, 1]$ is the correlation between features i and j .

The covariance combines the spread of each feature with how strongly the two features move together.

Standard Deviation and Correlation

Each diagonal entry of $\hat{\Sigma}$ is the variance of one feature:

$$\sigma_i^2 = \hat{\Sigma}_{ii} = \frac{1}{n-1} \sum_{k=1}^n (x_k^{(i)} - \bar{x}_i)^2.$$

The correlation divides the covariance by both standard deviations:

$$\rho_{ij} = \frac{\hat{\Sigma}_{ij}}{\sigma_i \sigma_j} \quad \Rightarrow \quad \hat{\Sigma}_{ij} = \sigma_i \sigma_j \rho_{ij}.$$

The correlation measures the direction and strength of the linear relationship on a scale from -1 to 1 . In the lower row of the figure, scaling the covariance matrix by four doubles each standard deviation and leaves the correlation unchanged.

Covariance from the Data Matrix

Stack the samples as the rows of the data matrix $\mathbf{X} \in \mathbb{R}^{n \times m}$, so rows are samples and columns are features. Let $\mathbf{m} = [\bar{x}_1, \dots, \bar{x}_m]^\top$ hold the feature means and let $\mathbf{1} \in \mathbb{R}^n$ be a vector of ones. The centered data matrix is:

$$\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}\mathbf{m}^\top.$$

The **outer product** $\mathbf{1}\mathbf{m}^\top$ is an $n \times m$ matrix whose rows all equal $\mathbf{m}^\top$. In general, the outer product of $\mathbf{a} \in \mathbb{R}^n$ and $\mathbf{b} \in \mathbb{R}^m$ has entries $(\mathbf{a}\mathbf{b}^\top)_{ij} = a_i b_j$. The empirical covariance matrix is then:

$$\hat{\Sigma} = \frac{1}{n-1} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}.$$

This form computes every covariance at once, without computing the correlations first.

Covariance Matrix Properties

- **Elements:** the diagonal entries $\hat{\Sigma}_{ii}$ are the variances of the features and are never negative. The off-diagonal entries $\hat{\Sigma}_{ij}$, $i \neq j$, are the covariances between pairs of features.
- **Sign:** if $\hat{\Sigma}_{ij} > 0$, feature j tends to be above its mean when feature i is above its mean. If $\hat{\Sigma}_{ij} < 0$, feature j tends to be below its mean when feature i is above its mean. If $\hat{\Sigma}_{ij} = 0$, the features are uncorrelated: there is no linear relationship between them.
- **Symmetry:** $\hat{\Sigma}_{ij} = \hat{\Sigma}_{ji}$ for all i and j , which follows from the definition of covariance.
- **Positive semidefinite:** $\mathbf{v}^\top \hat{\Sigma} \mathbf{v} \geq 0$ for every vector $\mathbf{v} \in \mathbb{R}^m$.

Decomposing the Covariance Matrix

Because $\hat{\Sigma}$ is real, symmetric, and positive semidefinite, its eigenpairs $(\mathbf{v}_i, \lambda_i)$ have special properties. We can write:

$$\hat{\Sigma} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_m),$$

where $\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_m] \in \mathbb{R}^{m \times m}$ holds the eigenvectors as columns.

- **Orthonormal eigenvectors:** the eigenvectors can be chosen as mutually orthogonal unit vectors, so $\mathbf{V}^\top \mathbf{V} = \mathbf{I}$, the identity matrix.
- **Nonnegative eigenvalues:** the eigenvalues can be ordered as $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m \geq 0$.
- Each eigenvalue λ_i equals the **variance of the data** along the direction $\mathbf{v}_i$.

Principal Components

Projecting a sample's deviation from the mean onto $\mathbf{v}_i$ gives its i -th **principal component**:

$$y_i = \mathbf{v}_i^\top (\mathbf{x} - \bar{\mathbf{x}}).$$

Across the dataset, y_i has variance λ_i . The direction $\mathbf{v}_1$ captures the most variance, the direction $\mathbf{v}_2$ captures the second most, and so on.

Keeping the top k eigenvectors, $\mathbf{V}_k = [\mathbf{v}_1, \dots, \mathbf{v}_k]$, gives the best k -dimensional linear reconstruction of the data in the least-squares sense. The subspace they span **maximizes the retained variance** and **minimizes the reconstruction error**.

These eigenpairs are the foundation of PCA: the rows of the transformation matrix $\mathbf{P}$ are $\mathbf{v}_1^\top, \dots, \mathbf{v}_k^\top$.

From Eigendecomposition to SVD

The eigendecomposition works for square, symmetric matrices such as $\hat{\Sigma}$. The centered data matrix $\tilde{\mathbf{X}} \in \mathbb{R}^{n \times m}$ is usually rectangular, with more samples than features or the reverse.

The **singular value decomposition** factors any rectangular matrix. Applied to the centered data matrix $\tilde{\mathbf{X}}$, it gives the same principal components as the eigendecomposition of $\hat{\Sigma}$, so the two approaches are mathematically equivalent.

The SVD has become the workhorse of dimensionality reduction because it is more general, it is numerically stable, and it factors the data matrix directly.

Singular Value Decomposition

Given any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ of rank $r = r(\mathbf{A})$, the **full SVD** is the factorization:

$$\mathbf{A} = \mathbf{U} \mathbf{S} \mathbf{V}^\top.$$

- $\mathbf{U} \in \mathbb{R}^{m \times m}$ is orthogonal: $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_m$, the $m \times m$ identity matrix.
- $\mathbf{S} \in \mathbb{R}^{m \times n}$ is rectangular diagonal, with the **singular values** $\sigma_1 \geq \dots \geq \sigma_r > 0$ on its first r diagonal entries and zeros elsewhere. From here on, σ_i denotes a singular value.
- $\mathbf{V} \in \mathbb{R}^{n \times n}$ is orthogonal: $\mathbf{V}^\top \mathbf{V} = \mathbf{I}_n$, the $n \times n$ identity matrix.

The singular values are the square roots of the eigenvalues of $\mathbf{A}^\top \mathbf{A}$, so $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^\top \mathbf{A})}$. The number of nonzero singular values is the rank of $\mathbf{A}$, and $r(\mathbf{A}) \leq \min(m, n)$.

Full and Compact SVD

The full SVD stores all m left singular vectors and all n right singular vectors, even though only the first $r(\mathbf{A})$ correspond to nonzero singular values.

- **Pro:** the full factors give complete orthonormal bases for $\mathbb{R}^m$ and $\mathbb{R}^n$.
- **Con:** the memory requirement grows as $\mathcal{O}(m^2 + n^2)$, which is wasteful when $r(\mathbf{A}) \ll \min(m, n)$.

The **compact SVD** keeps only the first $r = r(\mathbf{A})$ singular vectors and singular values:

$$\mathbf{A} = \mathbf{U}_r \mathbf{S}_r \mathbf{V}_r^\top, \quad \mathbf{U}_r \in \mathbb{R}^{m \times r}, \quad \mathbf{S}_r \in \mathbb{R}^{r \times r}, \quad \mathbf{V}_r \in \mathbb{R}^{n \times r}.$$

The **thin SVD** sits in between: it keeps $\min(m, n)$ singular vectors on each side. Julia's `svd(...)` function returns the thin SVD by default.

PCA from the SVD

Let the centered data matrix $\tilde{\mathbf{X}} \in \mathbb{R}^{n \times m}$ have the full SVD $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$. Substituting into the covariance matrix and using $\mathbf{U}^\top\mathbf{U} = \mathbf{I}_n$ gives:

$$\hat{\Sigma} = \frac{1}{n-1} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} = \frac{1}{n-1} \mathbf{V} \mathbf{S}^\top \underbrace{\mathbf{U}^\top \mathbf{U}}_{\mathbf{I}_n} \mathbf{S} \mathbf{V}^\top = \mathbf{V} \left(\frac{\mathbf{S}^\top \mathbf{S}}{n-1} \right) \mathbf{V}^\top.$$

Here $\mathbf{S}^\top \mathbf{S}$ is diagonal with entries σ_i^2 , where σ_i are the singular values of $\tilde{\mathbf{X}}$. This is the eigendecomposition $\hat{\Sigma} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$:

- the right singular vectors of $\tilde{\mathbf{X}}$ are the **eigenvectors** of $\hat{\Sigma}$, and
- the eigenvalues are $\lambda_i = \sigma_i^2 / (n-1)$, taking $\sigma_i = 0$ for $i > r(\tilde{\mathbf{X}})$.

We can compute the principal directions and their variances from the SVD of $\tilde{\mathbf{X}}$ without forming $\hat{\Sigma}$.

Dimensionality Reduction with the SVD

The SVD writes any rectangular matrix as a weighted sum of rank-one blocks:

$$\mathbf{A} = \sum_{i=1}^{r(\mathbf{A})} \sigma_i \underbrace{(\mathbf{u}_i \otimes \mathbf{v}_i)}_{\mathbf{u}_i \mathbf{v}_i^\top}$$

where $\mathbf{u}_i$ and $\mathbf{v}_i$ are the i -th left and right singular vectors and σ_i are the ordered singular values. The outer product $\mathbf{u}_i \mathbf{v}_i^\top$ is a separable **rank-one block** of $\mathbf{A}$.

The blocks are weighted by their singular values, largest first. Truncating the sum keeps the blocks with the largest weights, which makes the SVD well suited to dimensionality reduction and data compression.

Eckart–Young Theorem

Truncate the sum after $k < r(\mathbf{A})$ terms:

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top.$$

For any other matrix $\mathbf{B}$ of rank at most k :

$$\|\mathbf{A} - \mathbf{A}_k\|_F \leq \|\mathbf{A} - \mathbf{B}\|_F, \quad \|\mathbf{A} - \mathbf{A}_k\|_2 \leq \|\mathbf{A} - \mathbf{B}\|_2,$$

where $\|\cdot\|_F$ is the Frobenius norm and $\|\cdot\|_2$ is the operator (spectral) norm.

Truncating the SVD is **provably optimal**: $\mathbf{A}_k$ is the closest rank- k matrix to $\mathbf{A}$, whether we measure closeness by the total squared error (the squared Frobenius norm) or by the maximum stretch (the spectral norm).

Example: Fun with Singular Value Decomposition

In this example, we decompose a grayscale photograph into rank-one frames and rebuild it from the first few.

1. Compute the SVD of the image matrix and check that $\mathbf{U}$ and $\mathbf{V}$ are orthogonal and that the factors multiply back to the image.
2. Compute the frames and measure the relative error, in the Frobenius norm, of the image built from the first k frames.
3. Add up the first few frames and check that the rank of the sum equals the number of frames.

The example does not center the image, so the first frame mostly holds the image's average brightness. The later frames add edges and finer detail.

Example: [Fun with Singular Value Decomposition](#)

Lab: SVD of a Platelet Metabolic Network

In [L7b](#), we construct a stoichiometric matrix from a human platelet metabolic model and use its SVD to examine:

- the **numerical rank** of the stoichiometric matrix,
- **matrix reconstruction** from the leading modes, and
- **conservation relations** and **balanced-flux directions**.

The same decomposition that compresses an image also reveals the structure of a reaction network.

Lab: [SVD of a platelet stoichiometric matrix](#)

Summary

We used matrix decompositions to find the most important directions in high-dimensional data.

Key takeaways

- **Covariance eigendecomposition.** The eigenvectors are the principal directions; the eigenvalues are the variance along each one.
- **Optimal low-rank approximation.** Keeping the largest singular values gives the closest matrix of a given rank.
- **PCA and the SVD.** The SVD of the centered data gives the same principal directions as the covariance eigendecomposition.

The example and lab apply these ideas to an image and a metabolic network.